

Bug Bounty Hunting: Low-Hanging Bugs Beginners Can Easily Find

A Starter's Guide to Vulnerabilities That Don't Require Deep Expertise

Monika

Published September 23, 2026

Free: No

Read on Freedium · original

Bug Bounty-GitBook@t.me/GitBook_s

Contents

1. IDOR (Insecure Direct Object Reference) 0

2. Open Redirect 0

3. Missing Rate Limiting 0

4. Sensitive Information Disclosure 0

5. CSRF (Cross-Site Request Forgery) 0

6. Clickjacking 0

7. Subdomain Takeover 0

8. Broken Access Control on Admin/Hidden Endpoints 0

9. Weak Password Reset Logic 0

10. HTML/Content Injection 0

11. Reflected XSS in Simple Parameters 0

12. Improper CORS Configuration 0

A Simple Methodology to Follow 0

Tools Worth Learning Early 0

Reporting Tips 0

If you're new to bug bounty hunting, don't jump straight into chasing complex vulnerabilities like RCE or intricate authentication bypasses. There are several bug classes that are commonly found, relatively easy to spot with the right approach, and great for building a strong foundation. Below are some bugs that make excellent starting points but always remember: only test on authorized programs (like HackerOne or Bugcrowd) with explicit scope, since testing without permission is illegal.

1. IDOR (Insecure Direct Object Reference)

This is one of the most common and beginner-friendly bugs. When an application uses a direct ID in a URL or request parameter (like user_id=123) without proper access control checks, you can simply change the ID to access another user's data.

- How to find it: Create your own account, note down the ID of any resource (profile, order, invoice), then try modifying that ID (increment/decrement) and see what happens. Also try this across two separate test accounts you control — request a resource belonging to Account A while logged in as Account B.
- Things to watch for: Don't just check read access — test write/delete requests too, since the impact is often much higher there. Also check IDs hidden in API responses, hidden form fields, and GraphQL queries, not just the visible URL.
- Common locations: Invoice/order downloads, profile pictures, chat/message threads, "view as" features, API endpoints like /api/user/{id}/details.

2. Open Redirect

Many apps redirect users after login, logout, or clicking a link, often via a redirect=, next=, or return_url= parameter. If the app doesn't validate this properly, you can point it to an external malicious site.

- How to find it: Look for parameters like redirect, url, next, return_to, continue in the URL, then try replacing the value with an external domain (e.g., evil.com). Also test bypass tricks like https://target.com.evil.com (https://target.com.evil.com), //evil.com, or /\evil.com.
- Things to watch for: Low severity alone, but can be chained with phishing or OAuth token theft for bigger impact — mention this chaining potential in your report to increase its perceived severity.

3. Missing Rate Limiting

Login pages, OTP verification, password reset, and coupon/referral code fields — many such endpoints don't limit the number of attempts, opening the door to brute-force and enumeration attacks.

- How to find it: Send the same request repeatedly (login, OTP, forgot-password) using Burp Suite's Intruder/Repeater and see if there's any blocking, delay, or CAPTCHA after several attempts.
- Things to watch for: Test rate limiting separately for IP-based limits and account-based limits — some apps block an IP after 5 tries but never lock the account itself, which is still exploitable via a botnet or proxy rotation.

4. Sensitive Information Disclosure

Sometimes internal error messages, API keys, debug info, or old backup files remain publicly accessible.

- How to find it: Check .git folders, .env files, .DS_Store, web.config, JS files (search for hardcoded API keys, internal URLs), and error stack traces triggered by malformed input.
- Things to watch for: Tools like GitHub dorking and JS file analysis (LinkFinder, JSFinder, SecretFinder) are very effective here. Also check public S3 buckets and exposed Swagger/API documentation pages (/swagger-ui.html, /api-docs).

5. CSRF (Cross-Site Request Forgery)

If a critical action (like changing an email or password) doesn't have a CSRF token, an attacker can trick a logged-in user into performing that action unknowingly by embedding a malicious auto-submitting form on another site.

- How to find it: Check whether sensitive forms carry a CSRF token, and if present, verify whether removing it (or reusing a token from a different session) still lets the request go through.
- Things to watch for: Many modern frameworks already have CSRF protection built in, so verify it's actually being enforced, not just present in the code. Also check if the SameSite cookie attribute is missing, since that alone can enable CSRF even where a token exists.

6. Clickjacking

If a site doesn't set the X-Frame-Options or CSP frame-ancestors header, it can be embedded inside an iframe, letting attackers trick users into clicking hidden buttons.

- How to find it: Try loading the target page inside an iframe on your own test HTML page and see if it renders.
- Things to watch for: Only impactful on pages that perform sensitive actions (like account settings, fund transfers, or "delete account") — reporting it on a static marketing page usually gets marked informational/won't-fix.

7. Subdomain Takeover

When a subdomain's DNS record (usually a CNAME) points to a third-party service (like AWS S3, GitHub Pages, Heroku, Shopify) that's no longer active, you can potentially claim that service yourself and serve your own content on the company's subdomain.

- How to find it: Enumerate subdomains (subfinder, amass, assetfinder), then check each one for a "not found," "no such bucket," or "domain not configured" error indicating an unclaimed service. The can-i-take-over-xyz GitHub repo lists fingerprints for dozens of services.
- Things to watch for: Verify carefully before reporting — false positives are extremely common here, so confirm the service is genuinely unclaimed before submitting.

8. Broken Access Control on Admin/Hidden Endpoints

Many apps hide admin panels or internal features by simply not linking to them, assuming "security through obscurity" is enough — but the endpoints still work if accessed directly.

- How to find it: Use directory brute-forcing tools (ffuf, dirsearch, gobuster) with common wordlists to discover paths like /admin, /internal, /debug, /api/v1/admin. Also inspect JS files for hardcoded route names.
- Things to watch for: Even if you find such a panel, only view enough to confirm the vulnerability — don't browse around or modify data, as that can cross into unauthorized access.

9. Weak Password Reset Logic

Password reset flows are a goldmine for beginners because they're often rushed during development.

- How to find it: Check if the reset token is predictable (sequential, timestamp-based), if it expires, if it's tied to the correct user/session, and if the token gets invalidated after use. Also test whether the reset link works even after you've already used it once.
- Things to watch for: Try changing the email parameter in the reset request while keeping your own token, or check response timing differences to confirm if an email exists in the system (user enumeration).

10. HTML/Content Injection

Not full XSS, but if user input gets reflected into the page without proper output encoding, you may be able to inject basic HTML (like fake login forms or links) even where script tags are filtered.

- How to find it: Try submitting simple HTML tags like **test** or in comment fields, profile bios, or search boxes, and check if they render instead of being displayed as plain text.
- Things to watch for: This is often a stepping stone toward finding reflected or stored XSS on the same input field, so treat it as a signal to dig deeper, not just report it in isolation.

11. Reflected XSS in Simple Parameters

While stored XSS is more severe, reflected XSS in search boxes, error pages, or URL parameters is often the easiest type for beginners to catch.

- How to find it: Use a basic payload like "> in every input field and URL parameter, then check the page source to see if it's reflected unencoded.
- Things to watch for: Try different contexts (inside an HTML attribute, inside a

12. Improper CORS Configuration

If a server reflects the Origin header back in Access-Control-Allow-Origin and also sets Access-Control-Allow-Credentials: true, any website can make authenticated requests to the API on the victim's behalf.

- How to find it: Send a request with a custom Origin header (e.g., Origin: evil.com) using Burp Suite and check the response headers.

- Things to watch for: This is only high-impact if it exposes sensitive data (like user profile info or tokens) — CORS misconfig on public, non-sensitive endpoints usually isn't worth reporting.

A Simple Methodology to Follow

1. Recon first: Map out the entire application — every feature, every role (admin/user/guest), every input field.
2. Test the boring stuff first: Password reset, account settings, file uploads, and redirects are where beginners find the most bugs, precisely because experienced hunters often skip past them chasing bigger targets.
3. Change one thing at a time: IDs, roles, headers, tokens — isolate variables so you understand exactly what triggers the bug.
4. Document as you go: Screenshot every step; you'll need it for your report.

Tools Worth Learning Early

- Burp Suite (Community Edition) — intercepting and modifying requests
- subfinder / amass — subdomain enumeration
- ffuf / dirsearch — directory and endpoint brute-forcing
- Nuclei — automated scanning with community-maintained templates
- LinkFinder / SecretFinder — extracting endpoints and secrets from JS files

Reporting Tips

- Write a clear, reproducible proof-of-concept with numbered steps.
- Explain the impact, not just the technical mechanism programs prioritize triage by business impact.
- Even a "low severity" bug gets taken seriously when the report is clean, well-structured, and includes a working PoC video or screenshots.

THANK YOU

BUY ME A COFFEE ☕

Monika (https://buymeacoffee.com/monikaak47) Hi, I'm Monika I break down real-world bug bounty reports into simple, practical write-ups.If my articles help you…

